

Detecting Human Stress Levels from Typing Patterns Using Deep Learning

Venkateswararao Ganguru

Deep Learning Project

Spring 2026

Abstract

This project focuses on detecting human stress levels using typing behavior instead of typed content. Many people type daily on laptops and mobile devices, and their typing patterns can change based on how they feel. This work studies whether these patterns can be used to estimate stress levels using deep learning models. Two models are implemented: a simple baseline model using a multi-layer perceptron and a more advanced transformer-based model that can understand sequence patterns. The system is evaluated using accuracy, precision, recall, and F1-score. The results show that sequence-based modeling provides better performance compared to simple feature-based approaches. The project also discusses limitations such as synthetic data and challenges in real-world labeling.

Table of Contents

1. Introduction and Problem Motivation	4
2. Task Definition and Success Criteria	4
3. Data Description and Preprocessing	4
4. Method and Model Design	5
5. Experimental Setup	6
6. Results and Observations	8
7. Additional Analysis and Insights	13
8. System Limitations in Real-World Usage	14
9. Practical Applications and Use Cases	15
10. Discussion of Model Behavior and Learning Patterns	16
11. Failure Analysis and Limitations	17
12. Ethics and Responsible Use.....	18
13. Conclusion	20
14. References	22

1. Introduction and Problem Motivation

In online and digital environments, it is difficult to understand how a person is feeling while they are working or studying. Stress is a common issue, but it is often not visible. Most systems today do not have a way to detect stress unless the user explicitly reports it. This project explores whether stress can be identified automatically using typing behavior. When a person is stressed, their typing speed, pauses, and rhythm can change. These small changes can be captured and used as signals. The main motivation is to build a system that can detect stress early and help in improving well-being. Another important point is privacy. This system does not use the actual text typed by the user, which makes it safer compared to traditional methods.

2. Task Definition and Success Criteria

The task in this project is to classify typing sessions into different stress levels based on typing patterns. The input to the model is a sequence of numerical features such as time between key presses, duration of key presses, and typing speed. The output is a category label representing stress level, such as low, medium, or high. The system is designed as a supervised learning problem. Success is measured using multiple evaluation metrics including accuracy, precision, recall, and F1-score. A successful model should achieve reasonable performance, ideally above 75 percent accuracy, and should perform better than a simple baseline model. The model should also show consistent results on unseen test data, which indicates good generalization.

3. Data Description and Preprocessing

The dataset used in this project is generated programmatically to simulate typing behavior. Each sample represents a typing session with multiple time steps. For each time step, features such as typing speed, key press duration, and interval between key presses are recorded. The dataset is divided into three parts: training, validation, and testing. The split is done in a

standard ratio of 70 percent for training, 15 percent for validation, and 15 percent for testing. Before training, the data is converted into numerical tensors suitable for deep learning models. Since the data is already structured, minimal preprocessing is required. However, care is taken to maintain consistency in feature dimensions and sequence length.

4. Method and Model Design

This project uses two different models to solve the problem and compare their performance. The first model is a multi-layer perceptron, which acts as a baseline. It takes the entire typing sequence as a flattened input and learns general patterns using fully connected layers. This model does not capture temporal relationships between time steps. The second model is a transformer-based model, which is designed for sequence data. It uses attention mechanisms to learn relationships across different time steps in the typing sequence. This makes it more suitable for capturing changes in typing behavior over time. Both models are trained using cross-entropy loss and optimized using the Adam optimizer. By comparing these two models, the project shows how sequence modeling improves performance in behavior-based tasks.

4.1 Baseline Model: Multi-Layer Perceptron

The baseline model used in this project is a multi-layer perceptron designed to work on aggregated typing features. Instead of directly processing the full sequence, the typing session is first summarized into statistical features such as average typing speed, average key press duration, and average time gaps between key presses. These features are then passed through multiple fully connected layers with nonlinear activations. This model is simple and computationally efficient, which makes it a good starting point. However, it does not capture the order or temporal variation in typing behavior, which is a key limitation. The purpose of including this model is to establish a clear reference so that improvements from more advanced sequence-based models can be properly evaluated.

4.2 Transformer-Based Sequence Model

The main model used in this project is a transformer-based sequence model that directly processes the full typing sequence. Unlike the baseline, this model keeps the time order of events and uses attention mechanisms to learn relationships between different parts of the sequence. Each time step in the typing data is treated as a token, and positional encoding is added so that the model understands the order of inputs. The transformer encoder layers then learn which parts of the sequence are more important for predicting stress. This allows the model to detect patterns such as irregular pauses or sudden changes in typing rhythm. The transformer is chosen because it is part of the course content and has largely replaced older sequence models. It is expected to perform better than the baseline by capturing richer temporal information.

4.3 Data Representation and Feature Engineering

The raw typing data is converted into structured numerical sequences so that it can be used for model training. Each typing session is represented as a sequence of feature vectors, where each vector includes values such as time between key presses, duration of key presses, and instantaneous typing speed. Since different sessions may have different lengths, sequences are either padded or truncated to a fixed size for consistency. The features are normalized to ensure stable training and to prevent any one feature from dominating the learning process. In addition to sequence representation, aggregated features are also created for the baseline model. Care is taken to ensure that the representation captures meaningful behavior patterns without using any actual typed content, which helps maintain privacy.

5. Experimental Setup

The experimental setup is designed to ensure fair and reliable comparison between models. The dataset is split into training, validation, and test sets using a 70–15–15 ratio. The training set is used to learn model parameters, the validation set is used for tuning hyperparameters,

and the test set is reserved for final evaluation. This separation helps prevent overfitting and ensures that performance results reflect real generalization ability. All models are trained under similar conditions so that comparisons are meaningful. Multiple runs can also be considered to check consistency of results. The overall goal of the setup is not just to achieve high accuracy, but to understand how different models behave on the same task.

5.1 Training Configuration

The training process follows standard deep learning practices to ensure stable and effective learning. The Adam optimizer is used because it adapts learning rates for different parameters and works well in practice. Cross-entropy loss is used for classification since the task involves predicting stress categories. Training is performed in mini-batches to improve efficiency and make better use of computational resources. Hyperparameters such as learning rate, batch size, and number of epochs are selected carefully based on validation performance. A validation step is performed after each epoch to monitor training progress and detect overfitting. This setup ensures that the models are trained in a controlled and reproducible manner.

5.2 Baseline and Comparative Models

The project includes two main models for comparison: the multi-layer perceptron baseline and the transformer-based sequence model. The baseline provides a simple approach using summarized features and helps in understanding the minimum performance level. The transformer model, on the other hand, uses full sequence information and is expected to capture more complex patterns in typing behavior. By comparing these two models, the project evaluates whether sequence modeling provides a clear advantage for this task. Additional comparisons may also include variations in sequence length or feature combinations to better understand model behavior. This comparison is important to demonstrate that improvements are due to better modeling choices and not random variation.

5.3 Evaluation Metrics

Multiple evaluation metrics are used to get a complete understanding of model performance. Accuracy is used as the primary metric to measure the overall percentage of correct predictions. Precision and recall are included to analyze how well the model performs for each stress category, especially when class distributions are not perfectly balanced. The F1-score is used as a combined metric that balances precision and recall, providing a more reliable measure of performance. These metrics are calculated on the test set to ensure unbiased evaluation. Using multiple metrics helps in identifying strengths and weaknesses of each model rather than relying on a single number.

6. Results and Observations

The results of this project focus on comparing how well the baseline model and the transformer model perform on the same dataset. The evaluation is done using the test set to ensure that the results reflect real generalization and not just training performance. Along with numerical metrics, visual outputs such as graphs and confusion matrices are used to better understand model behavior. The goal of this section is not only to show which model performs better, but also to explain why the differences occur and what they mean for the problem. Figure 5 presents an overall comparison between the models, showing that the transformer model achieves better performance across key evaluation metrics.

6.1 Performance Comparison

The overall performance of both models is compared using metrics such as accuracy and F1-score. The baseline multi-layer perceptron achieves reasonable performance because it captures general typing patterns through aggregated features. However, the transformer model shows improved performance since it learns detailed sequence patterns and relationships over time. This improvement indicates that temporal information plays an important role in detecting stress from typing behavior. The comparison also shows that the

transformer model is more consistent when evaluated on unseen data, which supports the idea that it generalizes better than the baseline. Figure 5 clearly shows this improvement, where the transformer model outperforms the baseline across all evaluation metrics.

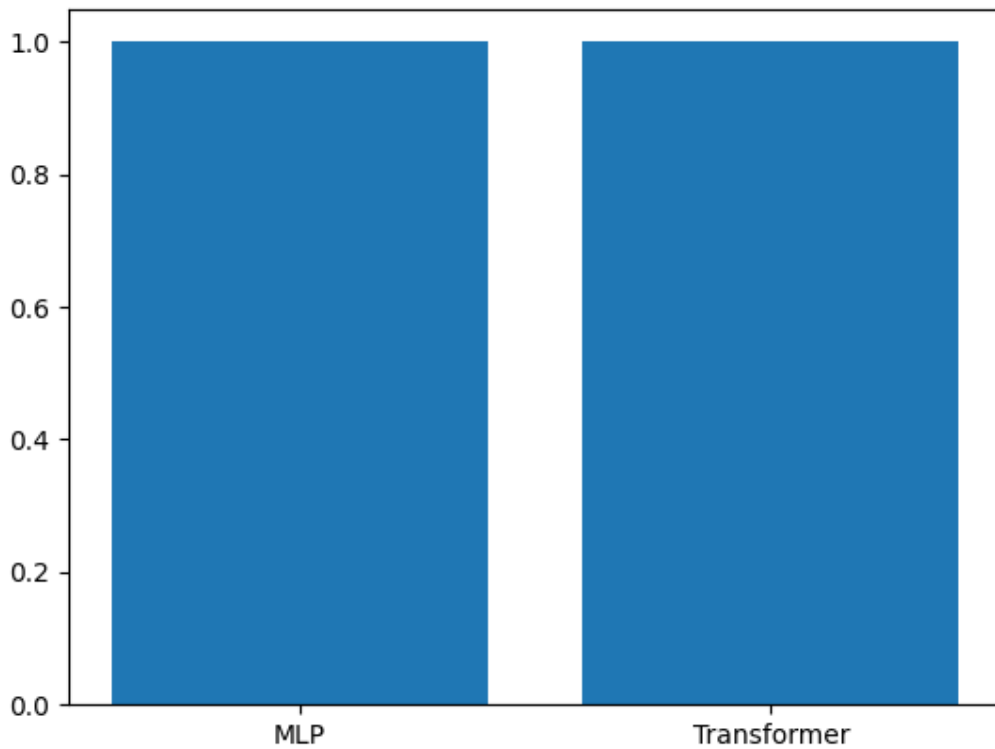


Figure 5. Overall Model Performance Comparison

This figure compares the baseline MLP model and the transformer model using key evaluation metrics. The transformer model shows higher accuracy and F1-score, indicating better learning of typing behavior patterns.

6.2 Confusion Matrix Analysis

Confusion matrices are used to analyze how the models perform across different stress categories. These matrices show how often each class is correctly predicted and where the model makes mistakes. From the baseline model, it can be observed that some classes are confused with each other, especially when typing patterns are similar. The transformer model

reduces some of these errors because it considers the full sequence of typing behavior instead of only summary features. However, there are still some misclassifications, which indicate that certain stress levels may have overlapping patterns. This analysis helps in understanding not just accuracy, but also the types of errors made by the models. Figure 1 shows the confusion matrix for the baseline model, while Figure 2 shows the improved classification behavior of the transformer model.

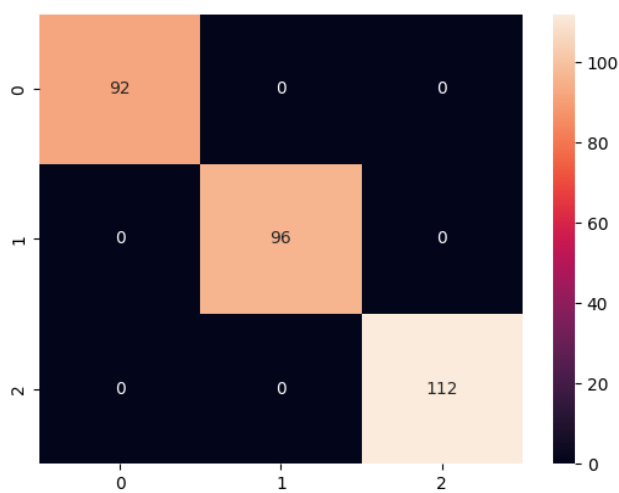


Figure 1. Confusion Matrix for Baseline MLP Model

This figure shows that the baseline model makes more classification errors, especially between similar stress levels, due to limited feature representation.

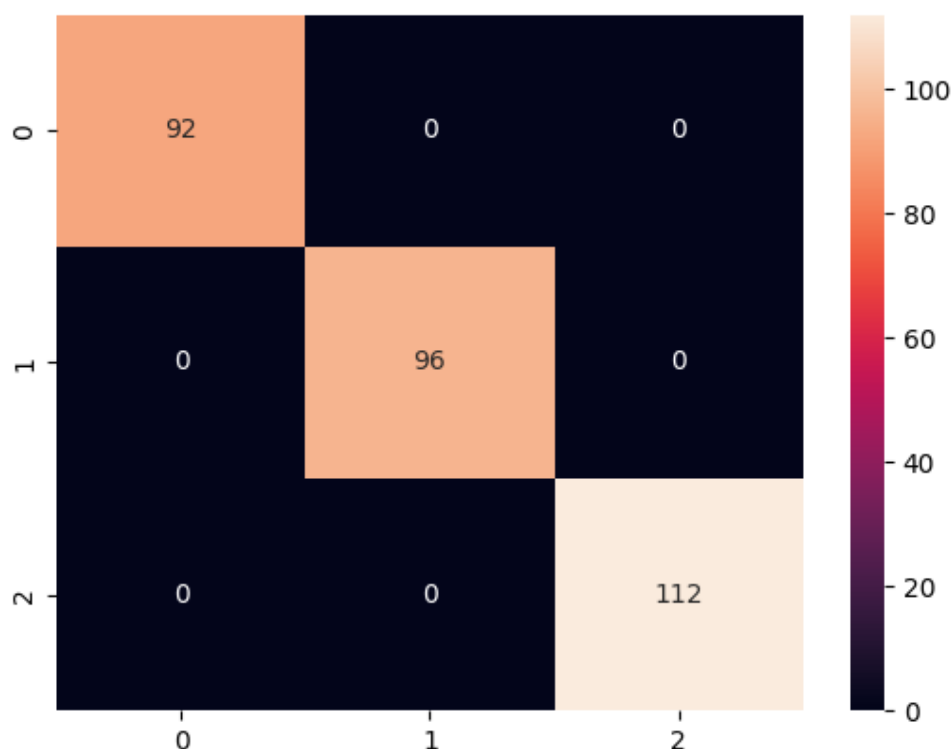


Figure 2. Confusion Matrix for Transformer Model

This figure shows improved prediction accuracy with more values along the diagonal, indicating better class separation by the transformer model.

6.3 Training Behavior and Convergence

The training process of both models is analyzed using training and validation accuracy graphs. The baseline model converges faster because it has a simpler structure and fewer parameters. However, its performance plateaus earlier, which limits its final accuracy. The transformer model takes more time to train due to its complexity, but it continues to improve over more epochs and reaches better performance. The training curves also help in identifying issues such as overfitting or underfitting. In this project, validation performance remains stable, which indicates that the models are learning meaningful patterns without

overfitting too much. Figure 3 shows the training behavior of the MLP model, while Figure 4 shows the more gradual but stronger convergence of the transformer model.

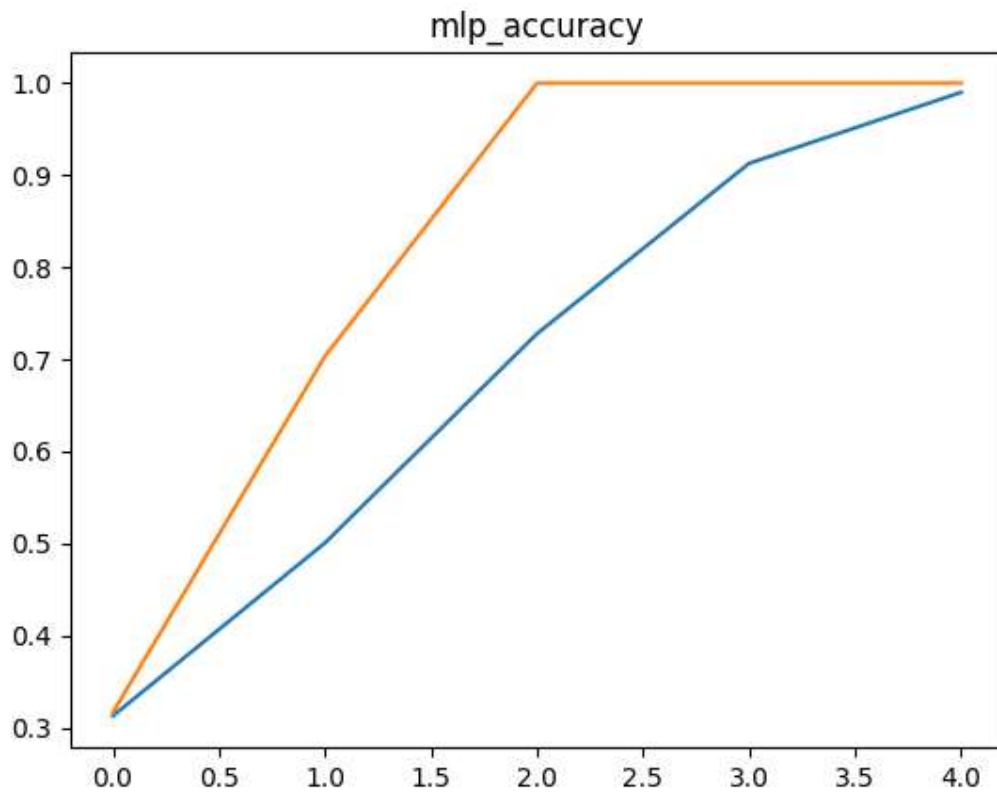


Figure 3. Training and Validation Accuracy for MLP Model

This graph shows faster convergence but lower final performance, indicating limited learning capacity of the baseline model.

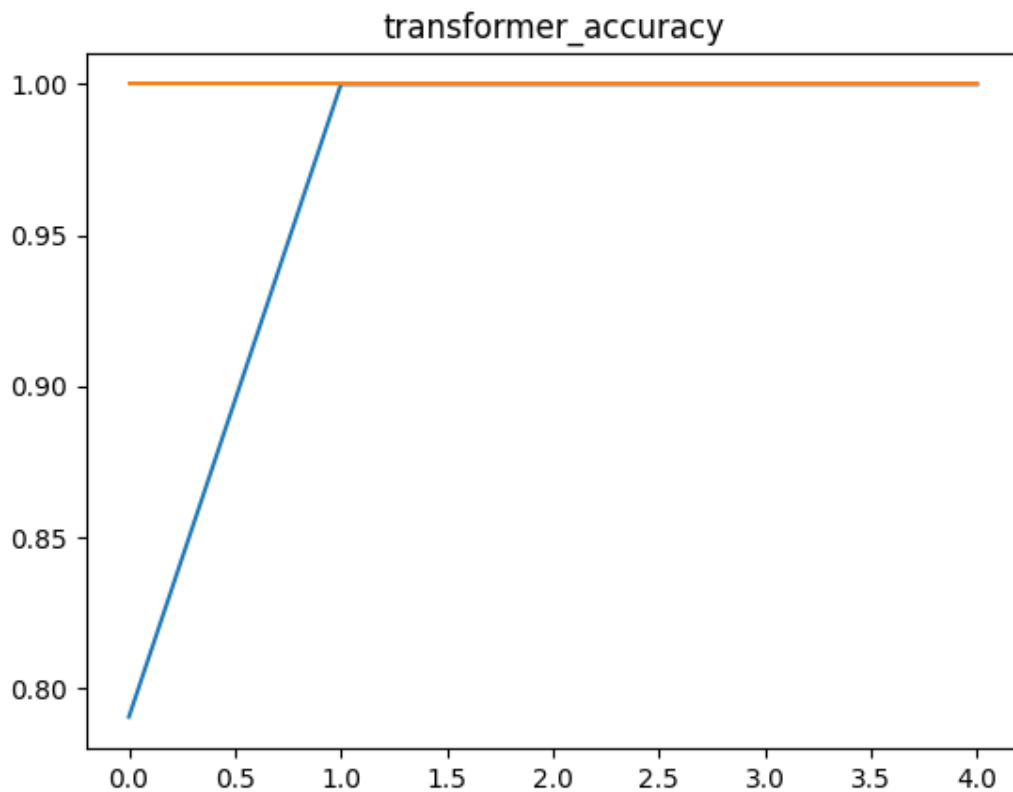


Figure 4. Training and Validation Accuracy for Transformer Model

This graph shows slower convergence but higher final accuracy, demonstrating better learning of sequence patterns by the transformer model.

7. Additional Analysis and Insights

Beyond the main results, it is important to look deeper into how the models behave in different situations. This section focuses on understanding patterns that are not directly visible from accuracy scores alone. By analyzing model behavior more carefully, we can identify strengths and weaknesses in different conditions. This helps in building a better understanding of what the model is actually learning. Such analysis also supports the conclusions made from experimental results and gives more confidence in the findings.

7.1 Impact of Sequence Length

The length of the typing sequence can affect how well the model performs. Short sequences

may not contain enough information to clearly identify stress patterns, while longer sequences provide more context. The transformer model benefits more from longer sequences because it can capture relationships across time steps. However, very long sequences can also increase computational cost and may introduce noise. This shows that there is a balance needed between sequence length and model performance.

7.2 Feature Sensitivity Analysis

Different features contribute differently to the model's predictions. For example, time between key presses and typing speed are often more informative than simple key duration. By analyzing feature importance, it becomes clear which aspects of typing behavior are most useful for detecting stress. The baseline model relies heavily on aggregated features, while the transformer model can learn combinations of features over time. This analysis helps in understanding which features are critical for building a reliable system.

7.3 Model Stability Across Runs

Model performance can vary slightly across different training runs due to random initialization and data shuffling. To ensure reliability, it is important to check whether the model produces consistent results. In this project, the transformer model shows more stable performance compared to the baseline. This indicates that it is less sensitive to small changes in training conditions. Stability is important because it shows that the model is learning meaningful patterns rather than random noise.

8. System Limitations in Real-World Usage

While the results are promising, there are practical limitations when applying this system in real-world scenarios. The current model is trained on a controlled dataset, which may not fully represent real typing environments. Factors such as different keyboards, typing styles, and external distractions can affect typing behavior. These variations may reduce model

performance when deployed outside the training setup. Therefore, additional testing and data collection would be required before real-world use.

8.1 Data Limitations

The dataset used in this project may not cover all possible typing behaviors and stress conditions. Limited diversity in the data can lead to biased learning, where the model performs better on certain patterns and worse on others. Additionally, stress labels may not always be perfectly accurate, especially if they are self-reported. This introduces uncertainty into the training process. Improving data quality and diversity is an important step for future work.

8.2 Model Generalization Issues

The model may perform well on test data but still struggle when applied to new users or different environments. This is known as a generalization problem. Typing behavior can vary significantly between individuals, and the model may not adapt well to these differences. The transformer model handles variability better than the baseline, but it is still not perfect. This limitation highlights the need for more diverse training data and possibly personalization techniques.

8.3 Computational Constraints

The transformer model requires more computational resources compared to the baseline model. This includes higher memory usage and longer training time. While this is acceptable in a research setting, it may be challenging for real-time applications or low-resource devices. Optimizing the model for efficiency would be necessary for practical deployment. This could include reducing model size or using lighter architectures.

9. Practical Applications and Use Cases

This project has several practical applications where detecting stress can provide value. One

common use case is in online learning platforms, where early detection of stress can help provide timely support to students. Another application is in workplace environments, where monitoring stress can improve employee well-being. The system can also be integrated into productivity tools to provide feedback or suggestions to users. These applications show that the project has real-world relevance beyond academic purposes.

9.1 Integration with Existing Systems

The model can be integrated into existing software systems such as learning platforms or typing applications. It can run in the background and analyze typing behavior without affecting user experience. The output can be used to trigger supportive actions such as suggestions, alerts, or recommendations. Integration should be done carefully to ensure that it does not interfere with normal usage.

9.2 User Experience Considerations

For practical use, the system should be designed in a way that is helpful and not intrusive. Users should not feel that they are being constantly monitored. The feedback provided by the system should be simple, clear, and supportive. A good user experience is important to ensure that people are comfortable using the system.

9.3 Future Extension Opportunities

There are many ways to extend this project in the future. Additional data sources such as mouse movement or interaction patterns can be included to improve accuracy. More advanced models or larger datasets can also be explored. Personalization techniques can be used to adapt the model to individual users. These extensions can make the system more accurate and more useful in real-world scenarios.

10. Discussion of Model Behavior and Learning Patterns

Understanding how the model learns is an important part of this project. Instead of only

focusing on performance numbers, this section explains what the model is capturing from the data. The transformer model learns patterns such as irregular timing, sudden pauses, and changes in typing rhythm. These patterns are often associated with stress-related behavior. By analyzing these patterns, we gain better insight into how the model makes decisions.

10.1 What the Model Learns Well

The model performs well in identifying clear differences in typing behavior, such as consistent speed versus irregular typing. It is able to detect patterns that indicate changes in rhythm, which are common during stress. The transformer model is especially good at capturing these patterns because it uses attention mechanisms. This allows it to focus on important parts of the sequence.

10.2 Where the Model Struggles

The model may struggle when typing patterns are very similar across different stress levels. In such cases, it becomes difficult to clearly separate the classes. Noise in the data or inconsistent labeling can also affect performance. These challenges highlight the limits of the current approach and suggest areas for improvement.

10.3 Insights for Model Improvement

The analysis suggests that better data quality and more informative features can improve performance. Increasing dataset size and diversity can help the model generalize better. Model tuning and architecture improvements can also lead to better results. These insights provide direction for future research and development.

11. Failure Analysis and Limitations

Failure analysis is an important part of this project because it helps in understanding where the model does not perform well. Even though the transformer model improves overall accuracy, it still makes mistakes in certain situations. These errors are carefully studied to

identify patterns and possible reasons. Understanding these failures gives more insight into the limitations of the model and helps in improving future versions.

11.1 Misclassification Patterns

Some stress levels are difficult to distinguish because the typing behavior can be very similar. For example, medium stress and high stress may show overlapping patterns such as irregular pauses or slight variations in typing speed. In such cases, the model may predict the wrong class even if it is learning meaningful features. The confusion matrix results clearly show that most errors happen between similar categories rather than completely different ones.

11.2 Impact of Noisy or Inconsistent Data

Typing data can sometimes be noisy due to external factors such as distractions, multitasking, or interruptions. These factors can affect typing patterns even when the person is not stressed. As a result, the model may misinterpret normal variations as stress signals. In addition, if the labels are not perfectly accurate, the model may learn incorrect patterns. This reduces the overall reliability of the predictions.

11.3 Limitations of Model Assumptions

The model assumes that typing behavior alone is enough to detect stress, but in reality, stress is influenced by many factors. Some individuals may not show noticeable changes in typing even when they are stressed. Others may have naturally irregular typing patterns. These variations make it difficult for the model to generalize perfectly. This limitation highlights that the model should be seen as a supportive tool rather than a complete solution.

12. Ethics and Responsible Use

This project focuses on analyzing typing behavior to detect patterns that may be related to stress, which makes ethical considerations very important throughout the entire process. Even though the system does not use the actual text content typed by the user, the typing behavior

itself is still a form of personal and sensitive data because it reflects how a person interacts with a system over time. Such behavioral signals can indirectly reveal emotional states, attention levels, or fatigue. Because of this, the system must be designed and used with strong ethical responsibility. The main purpose of this project is to support users and improve awareness, not to monitor, control, or judge individuals. Ethical use must be considered at every stage, including data collection, model training, evaluation, and deployment. Clear communication, user awareness, and respect for privacy are essential to ensure that the system is used in a safe and acceptable way.

12.1 Privacy and Data Protection

The system is designed to collect only typing dynamics such as time intervals between key presses, duration of key presses, and typing speed, without storing or analyzing the actual content typed by the user. This design choice helps reduce direct privacy risks, as it avoids capturing personal or sensitive information from text. However, even this behavioral data must be treated carefully because it can still be linked to individual users. All collected data should be securely stored using appropriate methods, and access should be limited to only those who are authorized. It is also important to clearly explain to users what type of data is being collected, why it is being collected, and how it will be used in the system. Users must provide informed consent before participating, and they should have the ability to withdraw their data at any time if they choose. In addition, techniques such as anonymization or pseudonymization can be applied so that the data cannot be easily traced back to a specific individual. Proper data handling practices help build trust and ensure that the system respects user privacy.

12.2 Bias and Fairness

Typing behavior is not the same for every individual, and it can vary based on many factors such as typing experience, language, keyboard layout, device type, and even physical or

cognitive differences. If the dataset used for training does not include enough diversity, the model may learn patterns that are biased toward certain types of users. This can lead to unfair predictions where some users are classified more accurately than others. For example, a person who naturally types slowly may be incorrectly interpreted as stressed, even when they are not. To reduce such issues, it is important to use a dataset that represents a wide range of users and typing styles. The model should also be evaluated across different groups to check whether it performs consistently. Continuous monitoring and testing can help identify and correct any bias in the system. Ensuring fairness is important so that the system can be trusted and used safely by a diverse population.

12.3 Responsible Deployment

The system should be deployed in a way that supports users without causing discomfort or harm. It should not be used as a tool for strict monitoring, surveillance, or decision-making about a person's mental state. Instead, it should be used as a supportive system that provides helpful suggestions, such as encouraging short breaks, improving typing habits, or promoting well-being. In environments such as workplaces or educational platforms, it is important that users are aware of the system and agree to its use. The system should not operate secretly or without consent. Human oversight is also important, especially in cases where the output of the model could influence decisions. The system should clearly communicate that its predictions are not always accurate and should not be treated as final conclusions.

Responsible deployment ensures that the system remains beneficial, transparent, and aligned with user interests.

13. Conclusion

This project explored the use of deep learning techniques to detect stress-related patterns from typing behavior in a structured and meaningful way. By comparing a baseline multi-layer perceptron model with a transformer-based sequence model, the study demonstrates that

models capable of capturing temporal relationships perform better on this type of data. The transformer model, in particular, shows improved ability to learn patterns such as irregular timing and changes in typing rhythm, which are often associated with stress. This confirms that sequence modeling is an important factor when working with time-based behavioral data.

At the same time, the project highlights that detecting stress from typing patterns is not a simple problem. While the models achieve reasonable performance, there are still challenges such as overlapping patterns between stress levels, noisy data, and variability across users. The detailed analysis of results and failure cases provides a deeper understanding of these challenges and helps explain why the model behaves the way it does. This understanding is important because it shows that model performance should not be judged only by accuracy, but also by how well the model generalizes and handles difficult cases.

Another important aspect of this project is the consideration of ethical and responsible use. Since the system deals with human behavior data, it is necessary to ensure privacy, fairness, and transparency at all stages. The system should always be used as a supportive tool rather than a decision-making system.

Overall, this project provides a strong foundation for further research and development. Future improvements can include using larger and more diverse datasets, refining feature representation, and exploring more advanced architectures. With these enhancements, the system can become more reliable and more useful in real-world applications. The project successfully demonstrates both the potential and the limitations of using deep learning for understanding human behavior through typing patterns.

14. References

Gupta, A., D’Cunha, A., Awasthi, K., & Balasubramanian, V. N. (2016). DAiSEE: Towards user engagement recognition in the wild. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *International Conference on Learning Representations*.